A03 — Definir modelo de API

1. Objetivo

Definir o contrato da API REST do SmartLog, estabelecendo os recursos, operações, requisições, respostas, regras de autorização e tratamento de erros necessários para atender às aplicações Flutter e Angular.

A API será implementada em Spring Boot e será responsável por intermediar todo acesso aos dados operacionais armazenados no Cloud Firestore.

A arquitetura definida no A02 estabelece que Flutter e Angular se comunicam com a API por HTTPS, utilizando `Authorization: Bearer <FIREBASE_ID_TOKEN>`. A API valida o token, identifica o usuário, seu perfil e seu `empresaId` antes de executar qualquer operação.

2. Princípios da API

A API seguirá os seguintes princípios:

- REST sobre HTTPS.
- Payloads em JSON.
- Autenticação via Firebase Authentication.
- Token enviado no header `Authorization`.
- Autorização baseada em RBAC.
- Isolamento obrigatório por `empresaId`.
- O `empresaId` será obtido a partir do contexto autenticado, e não confiado no payload enviado pelo cliente.
- Flutter e Angular não acessam diretamente o Firestore para operações protegidas.
- Respostas utilizam códigos HTTP semânticos.
- Os endpoints serão limitados aos casos de uso necessários para os perfis ADMIN, MOTORISTA e CLIENTE.

Os três perfis definidos pela arquitetura são ADMIN, MOTORISTA e CLIENTE.

---

3. Contexto de autenticação

Todas as operações protegidas exigem:

```
Authorization: Bearer <FIREBASE_ID_TOKEN>
```

O Spring Boot deve validar o token antes de executar a operação.

Após a validação, o backend deve disponibilizar internamente um contexto contendo, no mínimo:

```
{
  "uid": "firebase-user-id",
  "empresaId": "empresa-123",
  "role": "MOTORISTA"
}
```

Esse contexto não deve ser enviado pelo frontend como informação confiável.

O A02 determina que o backend extraia `empresaId` e perfil durante a validação do token e utilize essas informações para controlar o acesso ao Firestore.

---

4. Recursos da API

Os recursos prioritários são:

- Entregas
- Rotas
- Clientes
- Motoristas

Não será criado um CRUD genérico de usuários, empresas ou autenticação, pois essas responsabilidades pertencem ao Firebase Authentication e ao contexto de segurança da aplicação.

---

5. Entregas

As entregas representam os pedidos que precisam ser acompanhados e processados logisticamente.

O motorista pode consultar suas entregas e atualizar seu status. O cliente pode acompanhar suas próprias encomendas. O administrador possui visão operacional da empresa.

Os status previstos pela arquitetura são:

- `PENDENTE`
- `EM_ROTA`
- `ENTREGUE`

O A05 define explicitamente esses três estados para a operação do motorista.

5.1 Listar entregas

```
GET /api/v1/entregas
```

Autorização

- ADMIN: permitido.
- MOTORISTA: permitido, retornando somente entregas relacionadas às suas rotas.
- CLIENTE: permitido, retornando somente suas próprias entregas.

Parâmetros opcionais

```
status
rotaId
motoristaId
clienteId
```

O backend deve ignorar filtros que tentem ultrapassar o escopo de autorização do usuário.

Resposta

```
{
  "data": [
    {
      "id": "entrega-001",
      "codigo": "ENT-001",
      "clienteId": "cliente-001",
      "rotaId": "rota-001",
      "motoristaId": "motorista-001",
      "status": "EM_ROTA",
      "origem": "São Paulo - SP",
      "destino": "Santo André - SP",
      "criadaEm": "2026-08-19T10:00:00Z",
      "atualizadaEm": "2026-08-19T12:30:00Z"
    }
  ]
}
```

5.2 Consultar entrega

```
GET /api/v1/entregas/{id}
```

Autorização

- ADMIN: qualquer entrega da própria empresa.
- MOTORISTA: somente entrega vinculada ao seu escopo operacional.
- CLIENTE: somente sua própria entrega.

Resposta

```
{
  "id": "entrega-001",
  "codigo": "ENT-001",
  "clienteId": "cliente-001",
  "rotaId": "rota-001",
  "motoristaId": "motorista-001",
  "status": "EM_ROTA",
  "origem": "São Paulo - SP",
  "destino": "Santo André - SP",
  "criadaEm": "2026-08-19T10:00:00Z",
  "atualizadaEm": "2026-08-19T12:30:00Z"
}
```

5.3 Criar entrega

```
POST /api/v1/entregas
```

Autorização

- ADMIN: permitido.
- MOTORISTA: não permitido.
- CLIENTE: não permitido.

Request

```
{
  "clienteId": "cliente-001",
  "rotaId": "rota-001",
  "origem": "São Paulo - SP",
```

```
  "destino": "Santo André - SP"
}
```

O `empresaId` não faz parte do request.

O backend deve obtê-lo do contexto autenticado.

Response

```
201 Created
```

```
{
  "id": "entrega-001",
  "codigo": "ENT-001",
  "clienteId": "cliente-001",
  "rotaId": "rota-001",
  "status": "PENDENTE",
  "origem": "São Paulo - SP",
  "destino": "Santo André - SP"
}
```

5.4 Atualizar status da entrega

```
PATCH /api/v1/entregas/{id}/status
```

Autorização

- ADMIN: permitido.
- MOTORISTA: permitido somente para entregas sob sua responsabilidade.
- CLIENTE: não permitido.

Request

```
{
  "status": "ENTREGUE"
}
```

Response

```
{
  "id": "entrega-001",
  "status": "ENTREGUE",
  "atualizadaEm": "2026-08-19T13:00:00Z"
}
```

A operação é específica para mudança de estado porque o domínio não necessita de um `PUT` genérico de entrega.

---

6. Rotas

Rotas representam o agrupamento operacional utilizado para organizar as entregas e atribuí-las aos motoristas.

O ADMIN possui responsabilidade de gestão global das rotas, enquanto o MOTORISTA consulta as rotas atribuídas ao seu escopo.

6.1 Listar rotas

```
GET /api/v1/rotas
```

Autorização

- ADMIN: todas as rotas da empresa.
- MOTORISTA: somente rotas atribuídas a ele.
- CLIENTE: não permitido.

Response

```
{
  "data": [
    {
      "id": "rota-001",
      "nome": "Rota Zona ABC",
      "motoristaId": "motorista-001",
      "status": "EM_ANDAMENTO",
      "entregas": [
        "entrega-001",
        "entrega-002"
      ]
```

```
    }
  ]
}
```

6.2 Consultar rota

```
GET /api/v1/rotas/{id}
```

Autorização

- ADMIN: permitido para a própria empresa.
- MOTORISTA: somente se estiver vinculado à rota.
- CLIENTE: não permitido.

6.3 Criar rota

```
POST /api/v1/rotas
```

Autorização

- ADMIN: permitido.
- MOTORISTA: não permitido.
- CLIENTE: não permitido.

Request

```
{
  "nome": "Rota Zona ABC",
  "motoristaId": "motorista-001"
}
```

Response

```
201 Created
```

```
{
  "id": "rota-001",
  "nome": "Rota Zona ABC",
```

```
  "motoristaId": "motorista-001",
  "status": "PLANEJADA"
}
```

---

7. Motoristas

Motoristas representam os usuários responsáveis pela execução das rotas.

O ADMIN pode cadastrar motoristas, enquanto o MOTORISTA possui acesso apenas ao seu contexto operacional. O A05 determina que o cadastro realizado pelo administrador deve receber automaticamente o `empresaId` correspondente.

7.1 Listar motoristas

```
GET /api/v1/motoristas
```

Autorização

- ADMIN: permitido.
- MOTORISTA: não necessário.
- CLIENTE: não permitido.

Response

```
{
  "data": [
    {
      "id": "motorista-001",
      "nome": "João da Silva",
      "email": "joao@example.com",
      "ativo": true
    }
  ]
}
```

7.2 Consultar motorista

```
GET /api/v1/motoristas/{id}
```

Autorização

- ADMIN: permitido para motoristas da própria empresa.
- MOTORISTA: somente o próprio cadastro.
- CLIENTE: não permitido.

7.3 Criar motorista

```
POST /api/v1/motoristas
```

Autorização

- ADMIN: permitido.
- MOTORISTA: não permitido.
- CLIENTE: não permitido.

Request

```
{
  "nome": "João da Silva",
  "email": "joao@example.com"
}
```

Response

```
201 Created
```

```
{
  "id": "motorista-001",
  "nome": "João da Silva",
  "email": "joao@example.com",
  "ativo": true
}
```

O `empresaId` deve ser inserido pelo backend.

---

8. Clientes

Clientes representam os destinatários das entregas e possuem acesso limitado às suas próprias encomendas.

O A05 estabelece que o CLIENTE pode acompanhar o status e rastrear suas encomendas, com acesso restrito ao próprio histórico.

8.1 Listar clientes

```
GET /api/v1/clientes
```

Autorização

- ADMIN: permitido.
- MOTORISTA: não necessário.
- CLIENTE: não permitido.

8.2 Consultar cliente

```
GET /api/v1/clientes/{id}
```

Autorização

- ADMIN: permitido para clientes da própria empresa.
- CLIENTE: somente o próprio cadastro.
- MOTORISTA: não permitido.

8.3 Criar cliente

```
POST /api/v1/clientes
```

Autorização

- ADMIN: permitido.

Request

```
{
  "nome": "Cliente Exemplo",
  "email": "cliente@example.com",
  "telefone": "11999999999"
}
```

Response

```
201 Created
```

```
{
  "id": "cliente-001",
  "nome": "Cliente Exemplo",
  "email": "cliente@example.com",
  "telefone": "11999999999"
}
```

---

9. Matriz de autorização

Recurso	Operação	ADMIN	MOTORISTA	CLIENTE
Entregas	Listar	✓	✓*	✓*
Entregas	Consultar	✓	✓*	✓*
Entregas	Criar	✓	—	—
Entregas	Alterar status	✓	✓*	—
Rotas	Listar	✓	✓*	—
Rotas	Consultar	✓	✓*	—
Rotas	Criar	✓	—	—
Motoristas	Listar	✓	—	—
Motoristas	Consultar	✓	✓**	—
Motoristas	Criar	✓	—	—
Clientes	Listar	✓	—	—

Recurso	Operação	ADMIN	MOTORISTA	CLIENTE
Clientes	Consultar	✓	—	✓**
Clientes	Criar	✓	—	—

`*` O resultado deve ser filtrado pelo escopo do usuário.

`**` Somente o próprio recurso.

Essa matriz implementa o princípio definido no A05 de que ADMIN possui acesso amplo dentro da própria organização, MOTORISTA possui acesso operacional limitado e CLIENTE possui acesso exclusivamente aos próprios pedidos.

---

10. Tratamento de erros

Todos os erros devem possuir estrutura padronizada.

```
{
  "status": 400,
  "code": "VALIDATION_ERROR",
  "message": "Dados da requisição inválidos.",
  "timestamp": "2026-08-19T13:00:00Z",
  "path": "/api/v1/entregas"
}
```

Códigos HTTP

400 — Bad Request

Request inválido ou dados inconsistentes.

```
{
  "status": 400,
  "code": "VALIDATION_ERROR",
  "message": "O campo status é obrigatório."
}
```

401 — Unauthorized

Token ausente, inválido ou expirado.

```
{
  "status": 401,
  "code": "UNAUTHORIZED",
  "message": "Token de autenticação inválido ou ausente."
}
```

O A02 estabelece `401 Unauthorized` para tokens inválidos ou ausentes.

403 — Forbidden

Usuário autenticado, porém sem autorização para executar a operação.

```
{
  "status": 403,
  "code": "FORBIDDEN",
  "message": "Você não possui permissão para executar esta operação."
}
```

404 — Not Found

Recurso inexistente ou fora do escopo autorizado.

Por segurança, o backend pode tratar recursos pertencentes a outra empresa como `404`, evitando revelar sua existência.

409 — Conflict

Conflito de estado ou tentativa de criação de recurso duplicado.

500 — Internal Server Error

Falha inesperada no backend.

Detalhes internos, stack traces, credenciais ou informações sensíveis não devem ser retornados ao cliente.

---

11. Versionamento

A API utilizará versionamento no path:

```
/api/v1
```

Exemplo:

```
GET /api/v1/entregas
```

Alterações incompatíveis deverão resultar em uma nova versão da API.

---

12. Escopo final

A API definida neste documento contém somente operações necessárias para:

- autenticação e autorização dos usuários;
- gestão administrativa de entregas;
- gestão de rotas;
- cadastro e consulta de motoristas;
- cadastro e consulta de clientes;
- atualização operacional do status das entregas;
- acompanhamento das encomendas pelo cliente.

Não serão criados endpoints genéricos para:

- autenticação;
- gerenciamento de tokens;
- CRUD de `empresaId`;
- acesso direto ao Firestore;
- gerenciamento genérico de usuários do Firebase;
- operações de IA;
- processamento assíncrono.

A autenticação permanece sob responsabilidade do Firebase Authentication, enquanto o Worker é responsável pelo processamento em segundo plano e integração com serviços de IA, conforme definido no A02.

Critérios de aceite

- [x] Endpoints necessários definidos.
- [x] Contratos de request definidos.
- [x] Contratos de response definidos.
- [x] Tratamento de erros definido.
- [x] RBAC definido por recurso.
- [x] Isolamento por empresa considerado em todas as operações.
- [x] Escopo limitado às necessidades de Flutter e Angular.
- [x] Não foram adicionados endpoints de infraestrutura ou responsabilidades já atribuídas ao Firebase/Worker.